Event-Driven Architecture with Apache Kafka — Enterprise Implementation Guide

Microservices Event-Driven Architecture for Real-Time Order Processing System

Version: 2.0 **Date:** March 2026 **Classification:** Internal — Architecture Reference **Author:** Senior Distributed Systems Architect

---

Table of Contents

1. Executive Summary
2. Event-Driven Architecture Overview
3. Apache Kafka Deep Dive
4. Microservices Integration Strategy
5. Step-by-Step Implementation Guide
6. Code Walkthrough
7. Design Patterns (Outbox, Saga, CQRS, Event Sourcing)
8. Real-World Challenges & Solutions
9. Performance Tuning Guide
10. Security & Monitoring
11. Interview Preparation Questions
12. Future Scalability Considerations

---

1. Executive Summary

1.1 System Overview

The Real-Time Order Processing System is a production-grade microservices platform where **8 Kafka topics** interconnect **6 services** through asynchronous event-driven communication. The architecture processes order placement, inventory reservation, payment processing, and notification delivery — all without a single synchronous inter-service REST call in the critical path.

1.2 Key Architecture Metrics

Metric	Value
Total Services	8 (6 Kafka-enabled)
Kafka Topics	8
Event Models	8 (shared via common-events module)
Producers	5 services
Consumers	4 services, 6 consumer methods
Delivery Guarantee	At-Least-Once + Idempotency
Retry Strategy	@RetryableTopic (3 attempts, exponential backoff)
Dead Letter Topics	Auto-generated by @RetryableTopic

Idempotency	ProcessedEvent table (database-backed deduplication)
Partition Key Strategy	productId (ordering guarantee per product)

1.3 Topic Matrix

```
+--------------------+--------------------+----------------------------+---------------+
| Topic              | Producer           | Consumer(s)                | Key           |
+--------------------+--------------------+----------------------------+---------------+
| product-created    | Product Service    | Inventory Service          | productId     |
| product-events     | Product Service    | (Audit/Notification)       | eventId       |
| order-created      | Order Service      | Inventory Service          | productId     |
| order-cancelled    | Order Service      | Inventory Service          | productId     |
| inventory-updated  | Inventory Service  | Order Service, Payment Svc | orderId       |
| payment-success    | Payment Service    | Order Service, Notif. Svc  | orderId       |
| payment-failure    | Payment Service    | Order Service, Notif. Svc  | orderId       |
| audit-logs         | Order/User/Product | Notification Service       | eventId       |
+--------------------+--------------------+----------------------------+---------------+
```

1.4 Key Design Decisions

#	Decision	Rationale
1	Kafka over REST for inter-service communication	Loose coupling, fault tolerance, guaranteed delivery
2	At-Least-Once + Idempotency over Exactly-Once	Simpler, higher throughput, industry standard for e-commerce
3	productId as partition key	Ordering guarantee per product lifecycle
4	Shared common-events module	Single source of truth for event schemas across services
5	@RetryableTopic over manual retry	Spring-managed retry with exponential backoff + auto DLT
6	ErrorHandlingDeserializer	Prevents poison messages from blocking the consumer
7	Redis distributed locks	Prevents race conditions in stock reservation
8	Event-sourced audit trail via Kafka	Complete history of admin actions across all services

---

2. Event-Driven Architecture Overview

2.1 What is Event-Driven Architecture?

Event-Driven Architecture (EDA) is a software design paradigm where the production, detection, consumption, and reaction to **events** drives the flow of the system. An event represents a significant **state change** — "Order Created", "Payment Succeeded", "Stock Reserved".

```
TRADITIONAL (Request-Driven):
  User → Order Service → [REST call] → Inventory Service → [REST call] → Payment Service
  Problem: If Payment Service is down, the entire chain fails.
```

```
  Problem: User waits for ALL services to respond (latency = sum of all calls).

EVENT-DRIVEN (This Project):
  User → Order Service → [Kafka: order-created] → Returns immediately (200 OK)
                                      ↓
                              Inventory Service [consumes asynchronously]
                                      ↓
                              [Kafka: inventory-updated]
                                      ↓
                              Payment Service [consumes asynchronously]
                                      ↓
                              [Kafka: payment-success]
                                      ↓
                        Order Service + Notification Service
```

2.2 Synchronous vs Asynchronous Communication

```
+---------------------------+----------------------------+----------------------------+
| Aspect                    | Synchronous (REST/HTTP)    | Asynchronous (Kafka)       |
+---------------------------+----------------------------+----------------------------+
| Coupling                  | Tight — caller must know   | Loose — producer doesn't   |
|                           | about callee               | know consumers             |
+---------------------------+----------------------------+----------------------------+
| Availability              | Both must be running       | Consumer can be down;      |
|                           | simultaneously             | Kafka retains messages     |
+---------------------------+----------------------------+----------------------------+
| Latency (user-perceived)  | Sum of all service calls   | Only producer latency      |
|                           | in the chain               | (Kafka publish is <5ms)    |
+---------------------------+----------------------------+----------------------------+
| Scalability               | Limited by slowest service | Each service scales        |
|                           | in the call chain          | independently              |
+---------------------------+----------------------------+----------------------------+
| Error Handling            | Cascade failure (one down  | Isolated failures; Kafka   |
|                           | = all down)                | buffers during outages     |
+---------------------------+----------------------------+----------------------------+
| Data Consistency          | Strong consistency (2PC)   | Eventual consistency       |
+---------------------------+----------------------------+----------------------------+
| Debugging                 | Easy — synchronous stack   | Harder — distributed trace |
|                           | trace                      | needed (correlation IDs)   |
+---------------------------+----------------------------+----------------------------+
| Use Case                  | User-facing reads, simple  | Background processing,     |
|                           | CRUD, real-time validation | event pipelines, analytics |
+---------------------------+----------------------------+----------------------------+
```

2.3 Why Kafka Instead of REST Between Services?

In this order processing system, when a user places an order, five services must react:

With REST (Synchronous Chain):

```
User → Order Service
         → REST: Inventory Service (reserve stock)       150ms
```

```
            → REST: Payment Service (charge card)        500ms
               → REST: Notification Service (send email) 200ms
                  → REST: User Service (update history)  100ms
Total latency: 950ms (best case)
If ANY service is down: ENTIRE ORDER FAILS
```

With Kafka (Asynchronous Events — Current Implementation):

```
User → Order Service → Kafka (order-created)    5ms publish
     ← HTTP 200 "Order placed"                  INSTANT response

Background (all parallel):
  Inventory Service consumes → reserves stock → publishes inventory-updated
  Payment Service consumes inventory-updated → processes payment → publishes payment-success
  Notification Service consumes payment-success → creates notification
  Order Service consumes payment-success → updates order status
```

Benefits in this project:

1. **User response time: 5ms** (Kafka publish) vs 950ms (REST chain)
2. **If Payment Service is down:** Order still created. Kafka retains the event. Payment processes when service recovers.
3. **If Notification Service is slow:** Zero impact on order processing.
4. **Scaling:** Inventory Service can have 3 instances consuming from 3 partitions independently.

2.4 When NOT to Use Kafka

```
DON'T use Kafka for:

1. SYNCHRONOUS VALIDATION:
   "Is this product in stock?" → Use REST API (user needs immediate answer)
   In this project: GET /api/inventory/{productId}/check uses REST

2. SIMPLE CRUD:
   "Get user profile" → Use REST API (read operation, no state change)
   In this project: GET /api/users/profile uses REST

3. LOW-VOLUME OPERATIONS:
   Admin creates 5 products/day → Could use REST (but we chose Kafka for
   decoupling product-service from inventory-service)

4. STRONG CONSISTENCY REQUIRED:
   "Transfer $1000 between bank accounts" → Use 2PC or Saga with compensation
   In this project: We accept eventual consistency for order processing

5. PROTOTYPING / MVP:
   Kafka adds operational complexity (broker management, topic design,
   partition strategy). Start with REST, migrate to Kafka when scale demands.
```

2.5 Where Kafka Fits in This Architecture

```
+──────────────────────────────────────────────────────────────────────────────+
│                        API GATEWAY (port 8080)                               │
│                     JWT Validation + Routing                                 │
+────────────────┬──────────────┬──────────────┬──────────────┬──────────────+
                 │              │              │              │
          [REST/HTTP]    [REST/HTTP]    [REST/HTTP]    [REST/HTTP]
                 │              │              │              │
        +────────▼──+   +────────▼──+   +────────▼──+   +────────▼────────+
        │  Auth     │   │  User     │   │ Product   │   │   Order       │
        │ Service   │   │ Service   │   │ Service   │   │  Service      │
        │ (8081)    │   │ (8082)    │   │ (8083)    │   │  (8084)       │
        +───────────+   +─────┬─────+   +─────┬─────+   +──┬────────┬───+
                              │             │             │        │
            [NO KAFKA]        │     [KAFKA] │     [KAFKA]│        │[KAFKA]
            Auth uses         │             │             │        │
            only REST         │             ▼             ▼        │
                              │   +--------------+   +────────▼────────+
                              │   │  Kafka       │   │  Inventory      │
                              │   │  Cluster     │   │  Service        │
                              │   │  (9092)      │   │  (8085)         │
                              │   +-────┬────────+   +────────┬────────+
                              │         │                     │
                              │    [KAFKA]               [KAFKA]
                              │         │                     │
                              │   +─────▼────────+   +────────▼────────+
                              │   │ Notification  │   │  Payment       │
                              │   │ Service       │   │  Service       │
                              │   │ (8087)        │   │  (8086)        │
                              │   +──────────────+   +──────────────+
                              │                                │
                              +──────── [KAFKA: audit-logs] ─+
```

2.6 High-Level Event Flow Diagram

```
COMPLETE ORDER LIFECYCLE (Event-Driven):

USER PLACES ORDER
       │
       ▼
  ┌──────────────┐    order-created     ┌──────────────────┐
  │   Order      │ ──────────────────▶  │   Inventory      │
  │   Service    │                      │   Service        │
  │              │ ◀──────────────────  │                  │
  │  Updates     │  inventory-updated   │  Reserves stock  │
  │  status to   │                      │  (Redis lock)    │
  │  RESERVED    │                      └──────────┬───────┘
  └──────────────┘                                 │
       ▲                                   inventory-updated
       │                                           │
  payment-success                                  ▼
       │                                  ┌──────────────────┐
```

```
       │                              │   Payment        │
       │                              │   Service        │
       │                              │                  │
       └──────────────────────────────│  Processes       │
                                      │  payment         │
                                      └──────────┬───────┘
                                                 │
                                           payment-success
                                                 │
                                                 ▼
                                      ┌──────────────────┐
                                      │  Notification    │
                                      │  Service         │
                                      │                  │
                                      │  Creates user    │
                                      │  notification    │
                                      └──────────────────┘


COMPENSATING FLOW (Order Cancelled):

USER CANCELS ORDER
       │
       ▼
  ┌──────────────┐   order-cancelled    ┌──────────────────┐
  │   Order      │ ────────────────────▶ │   Inventory      │
  │   Service    │                       │   Service        │
  │              │  inventory-updated    │                  │
  │  Status:     │ ◀──────────────────── │  Restores stock  │
  │  CANCELLED   │    (RELEASED)         │  (idempotent)    │
  └──────────────┘                       └──────────────────┘
```

---

3. Apache Kafka Deep Dive

3.1 Kafka Internal Architecture

```
+──────────────────────────────────────────────────────────────────────────+
│                        KAFKA CLUSTER                                     │
│                                                                          │
│  +──────────────+    +──────────────+    +──────────────+               │
│  │   Broker 0   │    │   Broker 1   │    │   Broker 2   │               │
│  │              │    │              │    │              │               │
│  │  Topic A     │    │  Topic A     │    │  Topic A     │               │
│  │  Partition 0 │    │  Partition 1 │    │  Partition 2 │               │
│  │  (Leader)    │    │  (Leader)    │    │  (Leader)    │               │
│  │              │    │              │    │              │               │
│  │  Topic B     │    │  Topic A     │    │  Topic B     │               │
│  │  Partition 1 │    │  Partition 0 │    │  Partition 0 │               │
│  │  (Replica)   │    │  (Replica)   │    │  (Leader)    │               │
│  +──────────────+    +──────────────+    +──────────────+               │
│                                                                          │
│  +──────────────────────────────────────+                               │
```

```
|  |         ZooKeeper / KRaft            |                            |
|  |  - Broker registration               |                            |
|  |  - Topic configuration               |                            |
|  |  - Leader election                    |                            |
|  |  - Consumer group coordination        |                            |
|  +──────────────────────────────────────+                            |
+──────────────────────────────────────────────────────────────────────+
```

3.2 Core Concepts Explained

3.2.1 Broker

What: A Kafka broker is a single Kafka server instance that stores data and serves client requests. **Why:** Brokers form the backbone of Kafka. Multiple brokers form a cluster for fault tolerance and scalability. **How:** Each broker is identified by a unique ID. In this project, we run a single broker (sufficient for development). In production, use 3+ brokers for high availability.

This project's Docker setup:

```yaml
kafka:
  image: confluentinc/cp-kafka:7.5.0
  environment:
    KAFKA_BROKER_ID: 1                    # Unique broker identifier
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181  # ZooKeeper coordination
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1  # Single broker = 1 replica
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"    # Topics auto-created on first use
```

3.2.2 Topic

What: A named stream of records. Think of it as a "category" for messages. **Why:** Topics logically separate different event types. Each business domain gets its own topic. **How:** Producers write to topics; consumers read from topics. Topics are durable — messages persist based on retention policy.

This project's topic design:

```
DOMAIN-ORIENTED TOPIC NAMING:
  product-created    → Product lifecycle events
  order-created      → Order placement events
  order-cancelled    → Order cancellation events
  inventory-updated  → Inventory state change events
  payment-success    → Successful payment events
  payment-failure    → Failed payment events
  audit-logs         → Cross-service admin audit trail
  product-events     → General product events (CRUD)
  user-events        → User lifecycle events
```

3.2.3 Partition

What: A topic is divided into ordered, immutable logs called partitions. Each message in a partition gets a unique, sequential offset. **Why:** Partitions enable parallelism — multiple consumers can read from different partitions simultaneously. **How:** The partition is determined by the message key (hash of key % number of partitions).

```
Topic: order-created (3 partitions)

Partition 0: [OrderEvent{productId=1}] [OrderEvent{productId=4}] ...
Partition 1: [OrderEvent{productId=2}] [OrderEvent{productId=5}] ...
Partition 2: [OrderEvent{productId=3}] [OrderEvent{productId=6}] ...

All events for productId=1 → always Partition 0 (hash("1") % 3 = 0)
This GUARANTEES ordering for the same product across events.
```

3.2.4 Offset

What: A unique, sequential identifier assigned to each message within a partition. **Why:** Consumers track their position (which messages they've read) using offsets. This enables replay, restart, and exactly-once semantics. **How:** Consumers commit offsets to Kafka. On restart, they resume from the last committed offset.

```
Partition 0:
  Offset 0: {orderId:1, status:CREATED}
  Offset 1: {orderId:5, status:CREATED}
  Offset 2: {orderId:9, status:CREATED}  ← Consumer last committed here
  Offset 3: {orderId:13, status:CREATED} ← Consumer will read this next
```

3.2.5 Consumer Group

What: A set of consumers that cooperate to consume a topic. Each partition is consumed by exactly one consumer in the group. **Why:** Enables horizontal scaling — add more consumers to process more partitions in parallel. **How:** Kafka automatically assigns partitions to consumers in a group. If a consumer dies, its partitions are reassigned.

```
Topic: order-created (3 partitions)
Consumer Group: inventory-service-group

SCENARIO 1: Single Instance
  Instance A → consumes P0, P1, P2 (all partitions)

SCENARIO 2: Two Instances
  Instance A → consumes P0, P1
  Instance B → consumes P2

SCENARIO 3: Three Instances (optimal)
  Instance A → consumes P0
  Instance B → consumes P1
  Instance C → consumes P2

SCENARIO 4: Four Instances (one idle!)
  Instance A → consumes P0
  Instance B → consumes P1
  Instance C → consumes P2
  Instance D → IDLE (more consumers than partitions)

RULE: Max useful consumers = number of partitions
```

This project's consumer groups:

```
inventory-service-group  → consumes: product-created, order-created, order-cancelled
payment-service-group    → consumes: inventory-updated
order-service-group      → consumes: inventory-updated, payment-success, payment-failure
notification-service-group → consumes: payment-success, payment-failure, audit-logs
```

3.2.6 ZooKeeper / KRaft

What: ZooKeeper is the coordination service for older Kafka versions. KRaft (Kafka Raft) is the new built-in consensus protocol replacing ZooKeeper. **Why:** Kafka needs metadata management — which brokers are alive, which is the leader for each partition, consumer group state. **How:** This project uses ZooKeeper (Confluent Platform 7.5.0). For new installations, KRaft is recommended.

```yaml
# Current setup (ZooKeeper-based)
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000

# Migration path: KRaft (no ZooKeeper needed)
kafka:
  image: confluentinc/cp-kafka:7.5.0
  environment:
    KAFKA_PROCESS_ROLES: broker,controller     # Combined mode
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

3.2.7 Replication

What: Each partition can have multiple replicas across different brokers. One replica is the leader; others are followers. **Why:** Fault tolerance — if a broker dies, a follower is promoted to leader. **How:** Producers write to the leader. Followers replicate from the leader. `acks=all` ensures all in-sync replicas confirm the write.

```
Replication Factor = 3 (Production recommended)

Broker 0: Partition 0 (Leader)  → writes go here
Broker 1: Partition 0 (Follower) → replicates from leader
Broker 2: Partition 0 (Follower) → replicates from leader

If Broker 0 dies:
  Broker 1 promoted to leader → no data loss
  Minimum ISR (In-Sync Replicas) = 2 ensures durability

This project: replication-factor=1 (single broker dev setup)
Production: replication-factor=3, min.insync.replicas=2
```

3.3 Producer Flow

```
APPLICATION CODE → KafkaTemplate.send() → INTERCEPTORS → SERIALIZER
        │                                                    │
        │                                          ┌─────────┘
        │                                          ▼
```

```
        │                                   PARTITIONER
        │                                   (hash key % partitions)
        │                                         │
        │                                         ▼
        │                                   RECORD ACCUMULATOR
        │                                   (batches records per partition)
        │                                         │
        │                                         ▼
        │                                   SENDER THREAD
        │                                   (sends RecordBatch to broker)
        │                                         │
        │                                         ▼
        │                                   BROKER
        │                                   (writes to partition log)
        │                                         │
        │                                         ▼
        └──── callback.onComplete() ◄──────  ACK (success/failure)
              (CompletableFuture)
```

This project's producer configuration:

```yaml
producer:
  key-serializer: StringSerializer      # Keys are always Strings
  value-serializer: JsonSerializer      # Values are POJOs → JSON
  properties:
    enable.idempotence: true            # Prevents duplicate publishes
  acks: all                             # Wait for ALL replicas to confirm
  retries: 3                            # Retry on transient failures
```

3.4 Consumer Flow

```
BROKER → FETCH (poll records) → DESERIALIZER → INTERCEPTORS → APPLICATION
          │                                                        │
          │                                                        ▼
          │                                                 @KafkaListener
          │                                                 handleOrderCreated()
          │                                                        │
          │                                                        ▼
          │                                                 PROCESS MESSAGE
          │                                                 (business logic)
          │                                                        │
          │                     ┌── SUCCESS ──────────────────────┘
          │                     │                 │
          │                     ▼                 ▼ (FAILURE)
          │              COMMIT OFFSET     @RetryableTopic
          │              (auto/manual)     retry 1 (1s) → retry 2 (2s) → retry 3 (4s)
          │                                         │
          │                                         ▼ (ALL FAILED)
          │                                  DEAD LETTER TOPIC
          │                                  (manual investigation)
          └── POLL NEXT BATCH ◄──────── OFFSET COMMITTED
```

This project's consumer configuration:

```yaml
consumer:
  group-id: order-service-group
  auto-offset-reset: earliest               # Start from beginning if no offset
  key-deserializer: StringDeserializer
  value-deserializer: ErrorHandlingDeserializer  # Wraps delegate; handles poison pills
  properties:
    spring.deserializer.value.delegate.class: JsonDeserializer  # Actual deserializer
    spring.json.trusted.packages: "*"        # Trust all packages for deserialization
    spring.json.type.mapping: "paymentEvent:com.orderprocessing.events.PaymentEvent,..."
```

3.5 Delivery Semantics Comparison

```
+----------------+-------------------------------------------------------+
| Guarantee      | Description                                           |
+----------------+-------------------------------------------------------+
| AT-MOST-ONCE   | Message may be lost, never redelivered.               |
|                | Consumer commits offset BEFORE processing.            |
|                | If processing fails, message is lost.                 |
|                | Use: logging, metrics (non-critical)                  |
+----------------+-------------------------------------------------------+
| AT-LEAST-ONCE  | Message is never lost, may be redelivered.            |
| (THIS PROJECT) | Consumer commits offset AFTER processing.            |
|                | If consumer crashes after processing but before commit, |
|                | message is redelivered on restart.                    |
|                | REQUIRES: Idempotent consumer (ProcessedEvent table)  |
|                | Use: order processing, payments (with idempotency)    |
+----------------+-------------------------------------------------------+
| EXACTLY-ONCE   | Message processed exactly once. No loss, no duplicates. |
|                | Requires: Kafka Transactions + Idempotent Producer.   |
|                | Complex to implement, lower throughput.               |
|                | Use: financial transactions (critical)                |
+----------------+-------------------------------------------------------+

WHY THIS PROJECT USES AT-LEAST-ONCE:
  1. Simpler implementation (no Kafka transactions needed)
  2. Higher throughput (no transaction coordinator overhead)
  3. Idempotency via ProcessedEvent table handles duplicates
  4. Industry standard for e-commerce order processing
```

3.6 Message Ordering Strategy

```
THIS PROJECT'S APPROACH: Key-based ordering

Problem: Out-of-order processing can cause data corruption.
  Example: If "order-created" is processed before "product-created",
           inventory reservation fails (product doesn't exist yet).

Solution: Use productId as partition key for related topics.
```

```
  - product-created uses productId as key
  - order-created uses productId as key
  - order-cancelled uses productId as key

  All events for the same product → same partition → processed IN ORDER.

  product-created(p1) → order-created(p1) → order-cancelled(p1)
  All in Partition hash("1")%3 → GUARANTEED ORDER

LIMITATION:
  Ordering is only guaranteed WITHIN a partition, not across partitions.
  Events for productId=1 and productId=2 may be processed in any order.
  This is acceptable because they are independent business entities.
```

3.7 Event Schema Strategy

```
THIS PROJECT: JSON Serialization (Jackson)

Producer:  POJO → JsonSerializer → JSON bytes → Kafka
Consumer:  Kafka → JSON bytes → JsonDeserializer → POJO

Type Mapping (application.yml):
  Producer: spring.json.type.mapping: "orderEvent:com.orderprocessing.events.OrderEvent"
  Consumer: spring.json.type.mapping:
"paymentEvent:com.orderprocessing.events.PaymentEvent,..."

ADVANTAGES:
  ✓ Human-readable messages (easy debugging with kafka-console-consumer)
  ✓ No schema registry needed
  ✓ Easy to evolve (add fields with defaults)
  ✓ Native Spring Boot support

DISADVANTAGES:
  ✗ Larger message size (field names repeated)
  ✗ No schema enforcement (producer/consumer can diverge)
  ✗ No backward/forward compatibility guarantees

PRODUCTION ALTERNATIVE: Avro + Schema Registry
  Producer: POJO → AvroSerializer → Avro bytes → Kafka
  Schema Registry validates schema compatibility on every publish.
  Smaller messages, enforced contracts, compile-time type safety.
```

3.8 Topic Design Strategy

```
PRINCIPLES USED IN THIS PROJECT:

1. ONE TOPIC PER EVENT TYPE (not per service):
   ✓ order-created, order-cancelled (separate topics)
   ✗ order-events (single topic with type field)
   WHY: Consumers can subscribe to exactly what they need.
        Inventory doesn't care about general order events.
```

```
2. DOMAIN-ORIENTED NAMING:
   ✓ product-created, payment-success
   ✗ svc-product-evt-create, kafka-payment-ok
   WHY: Names describe business events, not technical details.

3. SEPARATE TOPICS FOR SUCCESS/FAILURE:
   ✓ payment-success, payment-failure (separate)
   ✗ payment-result (single topic with status field)
   WHY: Consumers can subscribe to only the events they care about.
        Order Service listens to both; Notification Service formats
        different messages for success vs failure.

4. AUDIT TOPIC AS CROSS-CUTTING CONCERN:
   audit-logs is published to by multiple services (Order, User, Product).
   Notification Service consumes and persists all audit entries.
```

---

4. Microservices Integration Strategy

4.1 Shared Event Model (common-events module)

WHY a shared module: All producers and consumers must agree on the event schema. Instead of duplicating DTOs in each service, a shared Maven module provides a single source of truth.

```
common-events/
  src/main/java/com/orderprocessing/
    events/                          # Business domain events
      OrderEvent.java                # orderId, userId, productId, quantity, status
      PaymentEvent.java              # orderId, transactionId, amount, status
      InventoryEvent.java            # orderId, productId, quantity, availableStock, status
      NotificationEvent.java         # userId, orderId, type, message
      ProductCreatedEvent.java       # productId, productName, initialStock
    commonevents/event/              # Cross-cutting events
      AuditLogEvent.java             # actorUsername, action, resourceType, resourceId
      UserEvent.java                 # User lifecycle events
      ProductEvent.java              # General product CRUD events
```

Every event has `eventId`:

```java
private String eventId;  // UUID — unique identifier for idempotency
```

This enables:

1. **Idempotent processing:** Consumer checks if eventId was already processed
2. **Distributed tracing:** Correlate events across services
3. **Audit trail:** Track which events triggered which actions
4. **Dead Letter Topic investigation:** Identify specific failed events

4.2 Service Communication Map

```
+──────────────────────────────────────────────────────────────────────────+
│                   KAFKA EVENT FLOW MAP                                    │
│                                                                          │
│  Product Service                                                         │
│    PRODUCES: product-created, product-events, audit-logs                 │
│    CONSUMES: (none)                                                      │
│                                                                          │
│  Order Service                                                           │
│    PRODUCES: order-created, order-cancelled, audit-logs                  │
│    CONSUMES: inventory-updated, payment-success, payment-failure         │
│                                                                          │
│  Inventory Service                                                       │
│    PRODUCES: inventory-updated, payment-failure                          │
│    CONSUMES: product-created, order-created, order-cancelled             │
│                                                                          │
│  Payment Service                                                         │
│    PRODUCES: payment-success, payment-failure                            │
│    CONSUMES: inventory-updated                                           │
│                                                                          │
│  Notification Service                                                    │
│    PRODUCES: (none)                                                      │
│    CONSUMES: payment-success, payment-failure, audit-logs                │
│                                                                          │
│  User Service                                                            │
│    PRODUCES: user-events, audit-logs                                     │
│    CONSUMES: (none)                                                      │
+──────────────────────────────────────────────────────────────────────────+
```

4.3 Consumer Group Isolation Strategy

```
WHY SEPARATE CONSUMER GROUPS?
  - inventory-service-group and order-service-group BOTH consume inventory-updated
  - With SAME group: only one service gets each message (load balancing)
  - With DIFFERENT groups: both services get EVERY message (fan-out)

  inventory-updated topic (fan-out):
    order-service-group   → ALL messages → updates order status
    payment-service-group → ALL messages → processes payment

  payment-success topic (fan-out):
    order-service-group        → ALL messages → updates order to PAYMENT_SUCCESS
    notification-service-group → ALL messages → creates success notification
```

---

5. Step-by-Step Implementation Guide

Step 1 — Kafka Setup with Docker

Docker Compose Configuration (from this project):

```yaml
# ZooKeeper — Kafka coordination service
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  container_name: order-system-zookeeper
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181       # Client connection port
    ZOOKEEPER_TICK_TIME: 2000         # Basic time unit (ms) for heartbeats
  ports:
    - "2181:2181"

# Kafka Broker
kafka:
  image: confluentinc/cp-kafka:7.5.0
  container_name: order-system-kafka
  depends_on:
    - zookeeper                        # Must start after ZooKeeper
  ports:
    - "9092:9092"                      # External client access
  environment:
    KAFKA_BROKER_ID: 1                 # Unique broker identifier
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181  # ZooKeeper connection
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
    #                           ↑ Internal (Docker)        ↑ External (host machine)
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1    # Single broker setup
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"      # Auto-create on first use
```

WHY two listeners?

- `PLAINTEXT://kafka:29092` — Used by services INSIDE Docker network (inter-container communication)
- `PLAINTEXT_HOST://localhost:9092` — Used by services running on the HOST machine (development)

Topic Creation (Manual — for production):

```bash
# Create topic with specific partition count and replication
kafka-topics --create \
  --bootstrap-server localhost:9092 \
  --topic order-created \
  --partitions 3 \
  --replication-factor 1

# List all topics
kafka-topics --list --bootstrap-server localhost:9092

# Describe topic configuration
kafka-topics --describe --bootstrap-server localhost:9092 --topic order-created
```

This project uses auto-create: `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"` means topics are created automatically when a producer first sends to them. For production, disable this and pre-create topics with explicit partition and replication configs.

Step 2 — Producer Service Implementation

2.1 Maven Dependencies

```
<!-- Spring Kafka -->
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
<!-- Common Events Module (shared DTOs) -->
<dependency>
    <groupId>com.orderprocessing</groupId>
    <artifactId>common-events</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

2.2 Producer Configuration (application.yml)

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092    # Kafka broker address
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.type.mapping: "orderEvent:com.orderprocessing.events.OrderEvent"
        enable.idempotence: true         # Prevent duplicate publishes
      acks: all                          # Wait for ALL replicas to confirm
      retries: 3                         # Retry on transient broker failures
```

Configuration parameters explained:

Property	Value	Why
`key-serializer`	StringSerializer	Partition keys are always strings (productId, orderId)
`value-serializer`	JsonSerializer	Event POJOs are serialized to JSON
`type.mapping`	Class mapping	Maps logical type name to Java class for deserialization
`enable.idempotence`	true	Ensures each message is written exactly once to the broker
`acks`	all	Producer waits until all in-sync replicas confirm the write
`retries`	3	Retry sending on transient network/broker errors

2.3 Producer Implementation Patterns

Pattern A: Full-featured Producer (OrderProducer)

```java
@Component
public class OrderProducer {
```

```java
    private static final Logger log = LoggerFactory.getLogger(OrderProducer.class);
    private static final String ORDER_CREATED_TOPIC = "order-created";
    private static final String ORDER_CANCELLED_TOPIC = "order-cancelled";

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public OrderProducer(KafkaTemplate<String, Object> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void sendOrderEvent(OrderEvent event) {
        log.info("Publishing OrderEvent to topic '{}': {}", ORDER_CREATED_TOPIC, event);

        // KEY = productId → ensures same-product events go to same partition
        CompletableFuture<SendResult<String, Object>> future = kafkaTemplate.send(
                ORDER_CREATED_TOPIC,
                String.valueOf(event.getProductId()),  // partition key
                event);

        // Async callback for logging success or failure
        future.whenComplete((result, ex) -> {
            if (ex != null) {
                log.error("Failed to send OrderEvent for orderId={}: {}",
                        event.getOrderId(), ex.getMessage());
            } else {
                log.info("OrderEvent sent | topic={} | partition={} | offset={}",
                        result.getRecordMetadata().topic(),
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
            }
        });
    }
}
```

WHY CompletableFuture callback:

- `KafkaTemplate.send()` is non-blocking — returns immediately
- The callback fires when the broker acknowledges the write
- On failure: log error for monitoring/alerting
- On success: log partition and offset for debugging/tracing

Pattern B: Simple Producer (PaymentProducer, InventoryProducer)

```java
@Component
public class PaymentProducer {
    private final KafkaTemplate<String, PaymentEvent> kafkaTemplate;

    public void sendPaymentEvent(String topic, PaymentEvent event) {
        log.info("Publishing PaymentEvent to '{}': {}", topic, event);
        kafkaTemplate.send(topic, String.valueOf(event.getOrderId()), event);
```

```
    }
}
```

WHY topic as parameter:

- Payment result can go to EITHER `payment-success` or `payment-failure`
- Decision is made at the call site based on business logic
- Single producer method, multiple destination topics

Pattern C: Multi-purpose Producer (ProductEventProducer)

```java
@Service
public class ProductEventProducer {
    private static final String PRODUCT_CREATED_TOPIC = "product-created";
    private final KafkaTemplate<String, Object> kafkaTemplate;

    // Domain event → specific consumer (Inventory Service)
    public void publishProductCreatedEvent(ProductCreatedEvent event) {
        kafkaTemplate.send(PRODUCT_CREATED_TOPIC,
                String.valueOf(event.getProductId()), event);
    }

    // General event → multiple consumers (Notifications, Analytics)
    public void publishProductEvent(ProductEvent event) {
        kafkaTemplate.send("product-events", event.getEventId(), event);
    }

    // Audit event → cross-cutting concern
    public void publishAuditLog(AuditLogEvent event) {
        kafkaTemplate.send("audit-logs", event.getEventId(), event);
    }
}
```

WHY separate methods for different event types:

- `product-created` → triggers inventory initialization (critical business logic)
- `product-events` → general notifications (non-critical)
- `audit-logs` → cross-cutting audit trail
- Different topics → different consumers → different SLAs

Step 3 — Consumer Service Implementation

3.1 Consumer Configuration (application.yml)

```yaml
spring:
  kafka:
    consumer:
      group-id: order-service-group        # Consumer group identity
      auto-offset-reset: earliest          # Start from beginning if no offset
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
```

```
      properties:
        spring.deserializer.value.delegate.class:
org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: "*"
        spring.json.type.mapping: >
          paymentEvent:com.orderprocessing.events.PaymentEvent,
          inventoryEvent:com.orderprocessing.events.InventoryEvent
```

Configuration parameters explained:

Property	Value	Why
`group-id`	service-specific	Identifies this service in the consumer group
`auto-offset-reset`	earliest	If no committed offset exists, read from the very beginning
`ErrorHandlingDeserializer`	wrapper	Catches deserialization errors (poison pills) without crashing
`JsonDeserializer`	delegate	Actual deserializer that converts JSON → POJO
`trusted.packages`	*	Trust all packages for JSON deserialization
`type.mapping`	class mapping	Maps incoming type headers to local Java classes

3.2 @RetryableTopic — Automatic Retry with Dead Letter Topic

```java
@RetryableTopic(
    attempts = "3",                                // 3 retry attempts
    backoff = @Backoff(delay = 1000, multiplier = 2),  // 1s → 2s → 4s
    dltStrategy = DltStrategy.FAIL_ON_ERROR        // Send to DLT after all retries
)
@KafkaListener(topics = "order-created", groupId = "inventory-service-group")
public void handleOrderCreated(OrderEvent event) {
    // Business logic here
}
```

HOW @RetryableTopic works internally:

```
ORIGINAL TOPIC: order-created
                    │
                    ▼
          Consumer processes message
                    │
              ┌─────┴─────┐
              │  SUCCESS?  │
              ├── YES ─────┤──→ Commit offset. Done.
              └──── NO ────┘
                    │
                    ▼
          RETRY TOPIC 1: order-created-retry-0 (wait 1s)
                    │
              ┌─────┴─────┐
```

```
            │  SUCCESS?   │
            ├── YES ──────┤──→ Done.
            └──── NO ─────┘
                  │
                  ▼
        RETRY TOPIC 2: order-created-retry-1 (wait 2s)
                  │
            ┌─────┴──────┐
            │  SUCCESS?   │
            ├── YES ──────┤──→ Done.
            └──── NO ─────┘
                  │
                  ▼
        RETRY TOPIC 3: order-created-retry-2 (wait 4s)
                  │
            ┌─────┴──────┐
            │  SUCCESS?   │
            ├── YES ──────┤──→ Done.
            └──── NO ─────┘
                  │
                  ▼
        DLT: order-created-dlt
        (Manual investigation required)
```

Spring auto-creates these topics:

- `order-created-retry-0`
- `order-created-retry-1`
- `order-created-retry-2`
- `order-created-dlt`

3.3 Consumer Implementation — Complex (InventoryConsumer)

```java
@Component
public class InventoryConsumer {
    private final InventoryService inventoryService;
    private final InventoryProducer inventoryProducer;

    // FLOW 1: Product Created → Initialize Inventory
    @RetryableTopic(attempts = "3", backoff = @Backoff(delay = 1000, multiplier = 2),
                    dltStrategy = DltStrategy.FAIL_ON_ERROR)
    @KafkaListener(topics = "product-created", groupId = "inventory-service-group")
    public void handleProductCreated(ProductCreatedEvent event) {
        log.info("Received ProductCreatedEvent: productId={}", event.getProductId());
        inventoryService.initializeInventory(
                event.getProductId(), event.getInitialStock(), event.getEventId());
    }

    // FLOW 2: Order Created → Reserve Stock
    @RetryableTopic(attempts = "3", backoff = @Backoff(delay = 1000, multiplier = 2),
                    dltStrategy = DltStrategy.FAIL_ON_ERROR)
    @KafkaListener(topics = "order-created", groupId = "inventory-service-group")
```

```java
    public void handleOrderCreated(OrderEvent event) {
        boolean reserved = inventoryService.reserveStock(
                event.getProductId(), event.getQuantity());

        InventoryEvent inventoryEvent = new InventoryEvent(
                UUID.randomUUID().toString(),
                event.getOrderId(), event.getUserId(), event.getProductId(),
                event.getQuantity(), 0,
                reserved ? "RESERVED" : "INSUFFICIENT_STOCK");

        // Route to appropriate downstream topic
        String topic = reserved ? "inventory-updated" : "payment-failure";
        inventoryProducer.sendInventoryEvent(topic, inventoryEvent);
    }

    // FLOW 3: Order Cancelled → Restore Stock (Idempotent)
    @RetryableTopic(attempts = "3", backoff = @Backoff(delay = 1000, multiplier = 2),
                    dltStrategy = DltStrategy.FAIL_ON_ERROR)
    @KafkaListener(topics = "order-cancelled", groupId = "inventory-service-group")
    public void handleOrderCancelled(OrderEvent event) {
        inventoryService.restoreStock(
                event.getProductId(), event.getQuantity(), event.getEventId());

        InventoryEvent inventoryEvent = new InventoryEvent(
                UUID.randomUUID().toString(),
                event.getOrderId(), event.getUserId(), event.getProductId(),
                event.getQuantity(), 0, "RELEASED");
        inventoryProducer.sendInventoryEvent("inventory-updated", inventoryEvent);
    }
}
```

WHY the Inventory Consumer is the most complex:

1. **Three event types:** product-created, order-created, order-cancelled
2. **Bidirectional communication:** Consumes events AND produces response events
3. **Conditional routing:** RESERVED → inventory-updated, INSUFFICIENT → payment-failure
4. **Idempotency:** Uses eventId from incoming event for deduplication
5. **Distributed locking:** Redis lock in reserveStock() prevents race conditions

3.4 Consumer Scaling

```
SCALING INVENTORY SERVICE:

3 partitions on each topic → maximum 3 concurrent consumers

Instance 1 (partitions: P0 for all topics):
  - product-created P0
  - order-created P0
  - order-cancelled P0

Instance 2 (partitions: P1):
  - product-created P1
```

```
  - order-created P1
  - order-cancelled P1

Instance 3 (partitions: P2):
  - product-created P2
  - order-created P2
  - order-cancelled P2

THROUGHPUT:
  1 instance: ~1000 events/sec
  3 instances: ~3000 events/sec (linear scaling)

TO SCALE BEYOND 3: Increase partition count → add more instances
  kafka-topics --alter --topic order-created --partitions 6
  Now 6 instances can consume in parallel.
```

Step 4 — Transaction Management

4.1 Database + Kafka Consistency Problem

```
PROBLEM: Dual Write
  Order Service must:
  1. Save order to MySQL (order_db)
  2. Publish OrderEvent to Kafka

  What if step 1 succeeds but step 2 fails?
  → Order saved but no event published → downstream services never know
  → INCONSISTENCY

  What if step 2 succeeds but step 1 fails?
  → Event published but no order in DB → downstream services process phantom order
  → INCONSISTENCY
```

4.2 Saga Pattern — Choreography (Current Implementation)

```
THIS PROJECT USES CHOREOGRAPHY-BASED SAGA:

Each service makes local decisions and publishes events.
No central coordinator — services react to events independently.

ORDER PLACEMENT SAGA:
  1. Order Service: Save order (status=CREATED) → Publish order-created
  2. Inventory Service: Reserve stock → Publish inventory-updated (RESERVED/INSUFFICIENT)
  3. Order Service: Update status (INVENTORY_RESERVED/CANCELLED)
  4. Payment Service: Process payment → Publish payment-success/failure
  5. Order Service: Update status (PAYMENT_SUCCESS/PAYMENT_FAILED)
  6. Notification: Create user notification

COMPENSATING ACTIONS:
  If Inventory says INSUFFICIENT_STOCK:
    → Order cancelled, no payment processed
  If Payment FAILED:
```

```
    → Order status = PAYMENT_FAILED
    → User can retry payment
  If user cancels order:
    → order-cancelled event
    → Inventory restores stock (idempotent via ProcessedEvent)
```

4.3 Outbox Pattern (Production Enhancement)

```
PROBLEM WITH CURRENT APPROACH:
  @Transactional
  void placeOrder() {
      orderRepository.save(order);       // Step 1: DB write (transactional)
      kafkaTemplate.send("order-created", event);  // Step 2: Kafka write (NOT transactional)
  }
  // If the application crashes between Step 1 and Step 2, the event is lost.

OUTBOX PATTERN SOLUTION:
  @Transactional
  void placeOrder() {
      orderRepository.save(order);       // Step 1: DB write
      outboxRepository.save(new OutboxEvent("order-created", event));  // Step 2: DB write
  }
  // BOTH writes are in the SAME database transaction → atomic

  // Separate thread/service polls the outbox table and publishes to Kafka:
  @Scheduled(fixedDelay = 1000)
  void publishOutboxEvents() {
      List<OutboxEvent> pending = outboxRepository.findByPublishedFalse();
      for (OutboxEvent event : pending) {
          kafkaTemplate.send(event.getTopic(), event.getPayload());
          event.setPublished(true);
          outboxRepository.save(event);
      }
  }
```

```sql
CREATE TABLE outbox_events (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    topic       VARCHAR(100) NOT NULL,
    key_value   VARCHAR(100),
    payload     JSON NOT NULL,
    published   BOOLEAN DEFAULT FALSE,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_published (published)
);
```

Step 5 — Event Versioning Strategy

```
BACKWARD COMPATIBILITY (Add new fields with defaults):
  V1: { orderId, userId, productId, quantity, status }
  V2: { orderId, userId, productId, quantity, status, discount: 0.0 }
```

```
  ✓ V1 consumers can still deserialize V2 messages (ignore unknown fields)
  ✓ V2 consumers handle V1 messages (discount defaults to 0.0)

FORWARD COMPATIBILITY (Consumers handle future versions):
  Configure Jackson: DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES = false
  ✓ Old consumers ignore new fields they don't understand

BREAKING CHANGES (require migration):
  - Renaming fields (userId → customerId)
  - Changing types (String orderId → Long orderId)
  - Removing required fields

  MIGRATION STRATEGY:
  1. Create new topic (order-created-v2)
  2. Deploy new consumers for v2
  3. Deploy producers to publish to BOTH v1 and v2
  4. Migrate all consumers to v2
  5. Decommission v1 topic
```

---

6. Code Walkthrough

6.1 Complete Order Placement Flow — Event by Event

```
USER: POST /api/orders
{
  "productId": 1,
  "quantity": 2
}

═══════════════════════════════════════════════════════════════════
STEP 1: Order Service — Create Order + Publish Event
═══════════════════════════════════════════════════════════════════

  OrderController.placeOrder()
    → OrderService.placeOrder()
      → 1. Verify product exists (REST call to product-service, cached in Redis)
      → 2. Calculate total price
      → 3. Save order to order_db: {orderId=42, status=CREATED}
      → 4. Build OrderEvent:
           {
             eventId: "uuid-abc-123",
             orderId: 42,
             userId: 7,
             productId: 1,
             productName: "Laptop",
             quantity: 2,
             totalPrice: 1999.98,
             status: "CREATED",
             createdAt: "2026-03-03 12:00:00"
           }
      → 5. OrderProducer.sendOrderEvent(event)
```

```
          kafkaTemplate.send("order-created", key="1", value=event)
          // key="1" (productId) → hash("1") % 3 = partition X
      → 6. Return HTTP 200: "Order placed successfully"

═══════════════════════════════════════════════════════════════════
STEP 2: Inventory Service — Reserve Stock
═══════════════════════════════════════════════════════════════════

  [Kafka delivers order-created event to inventory-service-group]

  InventoryConsumer.handleOrderCreated(event)
    → 1. Log: "Received OrderEvent: orderId=42, productId=1, qty=2"
    → 2. InventoryService.reserveStock(productId=1, quantity=2)
         → Acquire Redis distributed lock: "lock:inventory:1"
         → Read current stock: SELECT stock FROM inventory WHERE product_id=1
         → stock = 50, requested = 2
         → 50 >= 2 → SUFFICIENT
         → UPDATE inventory SET stock = 48 WHERE product_id = 1
         → Release Redis lock
         → Return true
    → 3. Build InventoryEvent:
         {
           eventId: "uuid-def-456",
           orderId: 42,
           userId: 7,
           productId: 1,
           quantity: 2,
           availableStock: 0,
           status: "RESERVED"
         }
    → 4. InventoryProducer.sendInventoryEvent("inventory-updated", event)
    → 5. Log: "Inventory RESERVED for orderId=42"

═══════════════════════════════════════════════════════════════════
STEP 3: Order Service — Update Status to INVENTORY_RESERVED
═══════════════════════════════════════════════════════════════════

  [Kafka delivers inventory-updated event to order-service-group]

  InventoryStatusConsumer.handleInventoryUpdated(event)
    → status = "RESERVED"
    → switch case: RESERVED
    → updateOrderStatus(orderId=42, INVENTORY_RESERVED)
      → UPDATE orders SET status='INVENTORY_RESERVED', updated_at=NOW() WHERE id=42
    → Log: "Order 42 status updated to INVENTORY_RESERVED"

═══════════════════════════════════════════════════════════════════
STEP 4: Payment Service — Process Payment
═══════════════════════════════════════════════════════════════════

  [Kafka delivers inventory-updated event to payment-service-group]
```

```
  PaymentConsumer.handleInventoryUpdated(event)
    → Check: status == "RESERVED"? YES → proceed
    → PaymentService.processPayment(orderId=42, amount=99.99)
      → Simulate payment processing
      → Save payment record to payment_db
      → Return Payment{status=SUCCESS, transactionId="TXN-789"}
    → Build PaymentEvent:
      {
        eventId: "uuid-ghi-789",
        orderId: 42,
        userId: 7,
        amount: 99.99,
        transactionId: "TXN-789",
        status: "SUCCESS"
      }
    → PaymentProducer.sendPaymentEvent("payment-success", event)

═══════════════════════════════════════════════════════════════════
STEP 5: Order Service — Update Status to PAYMENT_SUCCESS
═══════════════════════════════════════════════════════════════════

  [Kafka delivers payment-success event to order-service-group]

  OrderStatusConsumer.handlePaymentSuccess(event)
    → updateOrderStatus(orderId=42, PAYMENT_SUCCESS)
    → Log: "Order 42 status updated to PAYMENT_SUCCESS"

═══════════════════════════════════════════════════════════════════
STEP 6: Notification Service — Create User Notification
═══════════════════════════════════════════════════════════════════

  [Kafka delivers payment-success event to notification-service-group]

  NotificationConsumer.handlePaymentSuccess(event)
    → Build Notification:
      {
        orderId: 42,
        userId: 7,
        type: "PAYMENT_SUCCESS",
        message: "Your payment of $99.99 for Order #42 was successful!
                  Transaction ID: TXN-789"
      }
    → repository.save(notification)
    → Log: "Notification saved for orderId=42"
```

6.2 Idempotency Implementation (ProcessedEvent Table)

```java
// ProcessedEvent Entity — tracks which events have been processed
@Entity
@Table(name = "processed_events", indexes = {
    @Index(name = "idx_event_id", columnList = "eventId", unique = true)
```

```java
})
public class ProcessedEvent {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String eventId;        // UUID from the incoming event

    @Column(nullable = false)
    private String eventType;      // PRODUCT_CREATED, ORDER_CANCELLED

    @Column(nullable = false)
    private LocalDateTime processedAt;
}
```

Usage in InventoryService:

```java
public void restoreStock(Long productId, int quantity, String eventId) {
    // IDEMPOTENCY CHECK: Skip if already processed
    if (processedEventRepository.existsByEventId(eventId)) {
        log.warn("Event {} already processed — skipping restore", eventId);
        return;
    }

    // Process the event
    Inventory inventory = inventoryRepository.findByProductId(productId)
            .orElseThrow();
    inventory.setStock(inventory.getStock() + quantity);
    inventoryRepository.save(inventory);

    // Mark as processed (prevents reprocessing on Kafka replay)
    processedEventRepository.save(new ProcessedEvent(eventId, "ORDER_CANCELLED"));
}
```

WHY this is critical:

```
WITHOUT idempotency:
  1. Order cancelled → stock restored +5 (stock: 100 → 105)
  2. Consumer crashes before committing offset
  3. Kafka redelivers the same event
  4. Stock restored AGAIN +5 (stock: 105 → 110) ← WRONG!

WITH idempotency (ProcessedEvent table):
  1. Order cancelled → eventId "abc-123" checked → NOT found → process
  2. Stock restored +5 (stock: 100 → 105), save ProcessedEvent("abc-123")
  3. Consumer crashes, Kafka redelivers
  4. eventId "abc-123" checked → FOUND → SKIP ← CORRECT!
```

---

7. Design Patterns

7.1 Choreography-based Saga (Implemented)

```
WHAT: Each service listens to events and decides its own actions.
      No central coordinator — services are autonomous.

IMPLEMENTATION IN THIS PROJECT:
  Order Service
    ↓ order-created
  Inventory Service
    ↓ inventory-updated (RESERVED)
  Payment Service
    ↓ payment-success
  Order Service (final status update)
  Notification Service (user notification)

ADVANTAGES:
  ✓ Loose coupling (no single point of failure)
  ✓ Each service owns its own logic
  ✓ Easy to add new services (just subscribe to topics)

DISADVANTAGES:
  ✗ Hard to track the overall saga state
  ✗ Compensating actions are scattered across services
  ✗ Debugging requires distributed tracing
```

7.2 Orchestration-based Saga (Alternative)

```
WHAT: A central "Saga Orchestrator" coordinates the workflow.

IMPLEMENTATION (alternative approach):
  OrderSagaOrchestrator:
    1. Send "reserve-stock" command → Inventory Service
    2. Wait for "stock-reserved" reply
    3. Send "process-payment" command → Payment Service
    4. Wait for "payment-processed" reply
    5. Send "notify-user" command → Notification Service

ADVANTAGES:
  ✓ Clear workflow visibility
  ✓ Easy to add compensation logic
  ✓ Centralized error handling

DISADVANTAGES:
  ✗ Orchestrator is a single point of failure
  ✗ Tighter coupling (orchestrator knows all services)
```

7.3 Event Sourcing

```
WHAT: Store state as a sequence of events, not as current state.
      Current state is derived by replaying events.
```

```
EXAMPLE FOR ORDERS:
  Event Store:
    1. OrderCreated(orderId=42, product=Laptop, qty=2)
    2. InventoryReserved(orderId=42)
    3. PaymentProcessed(orderId=42, txn=TXN-789)
    4. OrderShipped(orderId=42, trackingId=TRACK-001)

  Current State = replay(events 1-4) → Order{status=SHIPPED}

  THIS PROJECT: Does NOT use full event sourcing but follows the principle:
    - Kafka retains events → can replay to rebuild state
    - Audit log events provide a partial event journal
    - ProcessedEvent table acts as a projection checkpoint
```

7.4 CQRS (Command Query Responsibility Segregation)

```
WHAT: Separate the write model (commands) from the read model (queries).

THIS PROJECT'S IMPLICIT CQRS:
  WRITE SIDE (Commands):
    Order Service → saves to order_db → publishes order-created
    Inventory Service → updates inventory_db → publishes inventory-updated
    Payment Service → saves to payment_db → publishes payment-success

  READ SIDE (Queries):
    Notification Service → consumes events → builds notification_db (read optimized)
    AuditLog Consumer → consumes audit-logs → builds audit_log table (read optimized)

  The notification_db and audit_log table are READ-OPTIMIZED projections
  built from events. They can be rebuilt by replaying Kafka topics.
```

---

8. Real-World Challenges & Solutions

8.1 Message Duplication

```
PROBLEM:
  Consumer processes message, crashes before committing offset.
  Kafka re-delivers the same message on restart.

WHY IT HAPPENS:
  - At-least-once delivery guarantee
  - Network timeouts during offset commit
  - @RetryableTopic retries

HOW TO DETECT:
  - Duplicate records in database
  - ProcessedEvent table shows same eventId twice (if broken)
  - Monitoring: count(eventId) > 1
```

```
HOW TO SOLVE (This Project):
  - ProcessedEvent table with unique constraint on eventId
  - Check before processing: if exists → skip
  - Database unique constraints as last-resort safety net

BEST PRACTICE:
  Make ALL consumers idempotent. Assume every message may arrive more than once.
```

8.2 Message Ordering Issues

```
PROBLEM:
  order-created event processed BEFORE product-created event.
  Inventory tries to reserve stock for a product that doesn't exist yet.

WHY IT HAPPENS:
  - Different topics may have different partition counts
  - Consumer lag on one topic but not another
  - Rebalancing delays

HOW TO DETECT:
  - "Product not found" errors in inventory-service logs
  - Race condition during load testing

HOW TO SOLVE (This Project):
  - Use productId as partition key in BOTH product-created and order-created
  - Same key → same partition → same consumer → sequential processing
  - @RetryableTopic: if product not found, retry after 1s (may be in-flight)

BEST PRACTICE:
  Use consistent partition keys across related topics.
```

8.3 Consumer Lag

```
PROBLEM:
  Consumer falls behind — messages accumulate faster than they're processed.

WHY IT HAPPENS:
  - Slow database operations in consumer
  - Too few partitions for the throughput
  - Consumer crashes and recovers (has to catch up)

HOW TO DETECT:
  - kafka-consumer-groups --describe --group inventory-service-group
  - Monitor LAG column (should be near 0)
  - Prometheus metric: kafka_consumer_records_lag

HOW TO SOLVE:
  - Increase partitions → add more consumer instances
  - Optimize consumer processing (batch reads, bulk DB writes)
  - Increase consumer poll batch size: max.poll.records=500
```

8.4 Broker Failure

```
PROBLEM:
  Kafka broker dies — producers can't publish, consumers can't consume.

WHY IT HAPPENS:
  - Hardware failure, network partition, OOM

HOW TO DETECT:
  - Kafka broker health check fails
  - Producer sends fail with TimeoutException
  - Consumer poll returns empty/errors

HOW TO SOLVE:
  - Run 3+ brokers (cluster)
  - replication-factor=3, min.insync.replicas=2
  - acks=all on producers (waits for all replicas)
  - Enable unclean.leader.election.enable=false (prefer consistency over availability)

BEST PRACTICE:
  NEVER run a single Kafka broker in production.
  This project uses 1 broker (development). Production needs 3+.
```

8.5 Poison Messages

```
PROBLEM:
  A malformed message (wrong schema, corrupt JSON) crashes the consumer
  on every attempt → consumer is stuck, can't progress past this offset.

WHY IT HAPPENS:
  - Producer bug (wrong serializer, missing required fields)
  - Manual message injection (testing, admin tools)
  - Schema evolution without backward compatibility

HOW TO DETECT:
  - Consumer stuck at same offset for extended period
  - Deserialization exceptions in logs
  - Consumer lag increasing but no successful processing

HOW TO SOLVE (This Project):
  - ErrorHandlingDeserializer wraps the delegate deserializer
  - If deserialization fails, the error is handled gracefully (not thrown)
  - @RetryableTopic sends to DLT after exhausting retries
  - DLT messages can be investigated and replayed manually

BEST PRACTICE:
  ALWAYS use ErrorHandlingDeserializer in production.
```

8.6 Network Partition

```
PROBLEM:
  Consumer is connected to Kafka but can't reach the database.
  Message is consumed but processing fails.

HOW TO SOLVE:
  - @RetryableTopic retries with exponential backoff (1s → 2s → 4s)
  - If DB recovers within retry window: message processed successfully
  - If DB still down after all retries: message goes to DLT
  - After DB recovery: replay DLT messages manually

BEST PRACTICE:
  Configure retry backoff to be longer than typical network recovery time.
```

8.7 Data Loss Prevention

```
PRODUCER SIDE:
  acks=all            → All replicas must acknowledge
  retries=3           → Retry on transient failures
  enable.idempotence  → Prevent duplicate writes

BROKER SIDE:
  replication.factor=3       → 3 copies of each partition
  min.insync.replicas=2      → At least 2 replicas must confirm
  unclean.leader.election=false  → Don't promote out-of-sync replicas

CONSUMER SIDE:
  enable.auto.commit=false   → Commit AFTER processing, not before
  @RetryableTopic            → Retry failures instead of skipping
  DLT                        → Save failures for manual replay
```

8.8 Backpressure Handling

```
PROBLEM:
  Producer sends events faster than consumer can process them.

HOW TO HANDLE:
  1. Kafka naturally buffers (retention period: default 7 days)
  2. Scale consumers (add instances up to partition count)
  3. Increase consumer batch size: max.poll.records=500
  4. Optimize consumer logic (async DB writes, batch processing)
  5. Add more partitions for parallelism
```

8.9 Large Message Handling

```
PROBLEM:
  Kafka default max message size: 1MB. Large payloads fail.

HOW TO SOLVE:
  1. BEST: Store large data in S3/MinIO, send reference in Kafka event
     { eventId: "...", orderId: 42, invoicePdfUrl: "s3://bucket/invoice.pdf" }
```

```
  2. ALTERNATIVE: Increase max.message.bytes on broker and topic
     kafka-configs --alter --topic large-events \
       --add-config max.message.bytes=10485760  # 10MB

  3. COMPRESSION: Enable in producer
     spring.kafka.producer.properties.compression.type: snappy
```

8.10 Monitoring and Observability

```
CRITICAL METRICS TO MONITOR:

  1. Consumer Lag: Messages consumed vs messages produced
     kafka-consumer-groups --describe --group inventory-service-group

  2. Throughput: Messages per second per topic
     kafka.server:type=BrokerTopicMetrics,name=MessagesInPerSec

  3. Error Rate: Failed message processing
     Application logs + custom counter metrics

  4. DLT Queue Depth: Messages in Dead Letter Topics
     kafka-console-consumer --topic order-created-dlt --from-beginning

  5. Broker Health: Active controller, under-replicated partitions
     kafka-metadata.sh or JMX metrics
```

---

9. Performance Tuning Guide

9.1 Producer Tuning

```
KEY PRODUCER PROPERTIES:

batch.size = 16384 (16KB default)
  - Producer batches records destined for the same partition
  - Larger batch = fewer network calls = higher throughput
  - Trade-off: higher batch = higher latency per message
  - RECOMMENDATION: 32768 (32KB) for high-throughput producers

linger.ms = 0 (default)
  - How long to wait for batch to fill before sending
  - 0 = send immediately (low latency)
  - 5-10ms = wait to fill batch (high throughput)
  - RECOMMENDATION: 5ms for this project (balance latency/throughput)

buffer.memory = 33554432 (32MB default)
  - Total memory for all unsent batches
  - If full, send() blocks for max.block.ms
  - RECOMMENDATION: 67108864 (64MB) for bursty workloads
```

```
compression.type = none (default)
  - Options: none, gzip, snappy, lz4, zstd
  - snappy: best balance of speed and compression (recommended)
  - Reduces network I/O, increases CPU usage
  - RECOMMENDATION: snappy for JSON payloads (40-60% compression)

max.in.flight.requests.per.connection = 5 (default)
  - With idempotence enabled, this MUST be <= 5
  - Higher values increase throughput but risk reordering
  - RECOMMENDATION: 5 (default is optimal with idempotence)
```

```yaml
# Optimized producer configuration (application.yml)
spring:
  kafka:
    producer:
      properties:
        batch.size: 32768
        linger.ms: 5
        buffer.memory: 67108864
        compression.type: snappy
        enable.idempotence: true
        max.in.flight.requests.per.connection: 5
      acks: all
      retries: 3
```

9.2 Consumer Tuning

```
KEY CONSUMER PROPERTIES:

max.poll.records = 500 (default)
  - Maximum records returned in a single poll()
  - Higher = more records per batch = higher throughput
  - BUT must process within max.poll.interval.ms or rebalance
  - RECOMMENDATION: 100-500 depending on processing time

max.poll.interval.ms = 300000 (5 min default)
  - Maximum time between polls before consumer is considered dead
  - If processing takes longer, consumer is kicked from group
  - RECOMMENDATION: Set based on slowest expected processing time

fetch.min.bytes = 1 (default)
  - Minimum data to fetch from broker
  - Higher values reduce network calls but increase latency
  - RECOMMENDATION: 1024 (1KB) for low-latency requirements

session.timeout.ms = 45000 (45s default)
  - Time before consumer is considered dead if no heartbeat
  - RECOMMENDATION: 10000-30000 (10-30s) for faster failure detection

heartbeat.interval.ms = 3000 (3s default)
  - Frequency of heartbeat to coordinator
```

```
  - Must be < session.timeout.ms / 3
  - RECOMMENDATION: 3000 (default is good)

enable.auto.commit = true (default in Spring)
  - true: offsets committed automatically
  - false: manual commit after processing
  - RECOMMENDATION: true for most cases; false for exactly-once needs
```

9.3 Broker Tuning (Production)

```
num.partitions = 3 (this project)
  - Default partition count for auto-created topics
  - RULE: partitions = max(expected_throughput / per_consumer_throughput,
                           max_consumers_per_group)
  - RECOMMENDATION: 6-12 for medium workloads, 24+ for high throughput

log.retention.hours = 168 (7 days default)
  - How long to keep messages
  - RECOMMENDATION: 72-168 hours (3-7 days) for most use cases

log.retention.bytes = -1 (unlimited default)
  - Maximum log size before deletion
  - RECOMMENDATION: Set if disk space is limited

log.segment.bytes = 1073741824 (1GB default)
  - Size of each log segment file
  - RECOMMENDATION: default is usually fine

num.io.threads = 8 (default)
  - Threads for I/O operations
  - RECOMMENDATION: Match to number of disks

num.network.threads = 3 (default)
  - Threads for network operations
  - RECOMMENDATION: 4-8 for high-throughput clusters
```

9.4 Partition Count Strategy

```
CALCULATING OPTIMAL PARTITION COUNT:

Formula: P = max(T/Cp, T/Cc)
  T  = Target throughput (messages/sec)
  Cp = Throughput per producer partition
  Cc = Throughput per consumer partition

Example for this project:
  Target: 10,000 orders/sec
  Consumer throughput: ~3,000 msgs/sec per partition
  P = 10000 / 3000 ≈ 4 partitions (minimum)
  Recommended: 6 (headroom for scaling)
```

```
CURRENT (development): 3 partitions per topic
PRODUCTION RECOMMENDED: 6-12 partitions per topic
```

---

10. Security & Monitoring

10.1 Kafka Security (Production)

```
SECURITY LAYERS:

1. ENCRYPTION IN TRANSIT (SSL/TLS):
   - Encrypt all communication between producers, consumers, and brokers
   - Prevent eavesdropping on sensitive event data (payment amounts, user IDs)

   # Broker configuration
   listeners=SSL://kafka:9093
   ssl.keystore.location=/etc/kafka/secrets/kafka.keystore.jks
   ssl.keystore.password=${KAFKA_SSL_KEYSTORE_PASSWORD}
   ssl.key.password=${KAFKA_SSL_KEY_PASSWORD}
   ssl.truststore.location=/etc/kafka/secrets/kafka.truststore.jks

   # Spring Boot producer/consumer
   spring.kafka.properties.security.protocol=SSL
   spring.kafka.ssl.trust-store-location=classpath:truststore.jks
   spring.kafka.ssl.trust-store-password=${KAFKA_SSL_TRUSTSTORE_PASSWORD}

2. AUTHENTICATION (SASL):
   - SASL/PLAIN: Username/password (simplest)
   - SASL/SCRAM: Salted challenge-response (more secure)
   - SASL/Kerberos: Enterprise SSO integration

   # Producer configuration
   spring.kafka.properties.security.protocol=SASL_SSL
   spring.kafka.properties.sasl.mechanism=SCRAM-SHA-512

spring.kafka.properties.sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule
required \
     username="order-service" password="${KAFKA_PASSWORD}";

3. AUTHORIZATION (ACLs):
   - Control which services can read/write to which topics

   # Order Service: can WRITE to order-created, order-cancelled
   kafka-acls --add --allow-principal User:order-service \
     --operation Write --topic order-created --topic order-cancelled

   # Inventory Service: can READ from order-created, order-cancelled
   kafka-acls --add --allow-principal User:inventory-service \
     --operation Read --topic order-created --topic order-cancelled --group inventory-service-
group

   # Deny all others
```

```
   kafka-acls --add --deny-principal User:* \
     --operation All --topic order-created
```

10.2 Topic Retention & Compaction

```
RETENTION STRATEGIES:

1. TIME-BASED RETENTION (default):
   log.retention.hours=168  # Keep messages for 7 days
   Use for: Event streams (order-created, payment-success)
   After 7 days, old events are deleted.

2. SIZE-BASED RETENTION:
   log.retention.bytes=1073741824  # Keep last 1GB per partition
   Use for: High-volume topics with disk constraints

3. LOG COMPACTION:
   log.cleanup.policy=compact
   Keeps only the LATEST message for each key.
   Use for: State topics (user-events, product-events)

   Before compaction:
     key=user1 → {name: "Alice", status: "active"}   offset 0
     key=user1 → {name: "Alice", status: "blocked"}  offset 5
     key=user2 → {name: "Bob", status: "active"}     offset 3

   After compaction:
     key=user1 → {name: "Alice", status: "blocked"}  offset 5  ← latest only
     key=user2 → {name: "Bob", status: "active"}     offset 3
```

10.3 Monitoring Stack

```
RECOMMENDED MONITORING STACK:

1. PROMETHEUS + GRAFANA:
   - Kafka broker metrics (JMX → Prometheus exporter)
   - Spring Boot Actuator + Micrometer → Prometheus
   - Consumer lag, throughput, error rates
   - Custom dashboards per topic and consumer group

2. KAFKA CONSOLE TOOLS:
   # Monitor consumer group lag
   kafka-consumer-groups --bootstrap-server localhost:9092 \
     --describe --group inventory-service-group

   Output:
   TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
   order-created   0          1523            1525            2
   order-created   1          1498            1498            0
   order-created   2          1510            1510            0
```

```
3. SPRING BOOT ACTUATOR:
   management:
     endpoints:
       web:
         exposure:
           include: health,prometheus,kafka
     metrics:
       export:
         prometheus:
           enabled: true

4. ALERTING RULES:
   - Consumer lag > 1000 for 5 minutes → WARNING
   - Consumer lag > 10000 for 5 minutes → CRITICAL
   - DLT message count > 0 → IMMEDIATE ALERT
   - Broker under-replicated partitions > 0 → CRITICAL
```

10.4 Distributed Tracing with Correlation IDs

```
CURRENT IMPLEMENTATION:
  Every event has an eventId (UUID) — acts as correlation ID within a single event.

PRODUCTION ENHANCEMENT — Trace ID across the entire saga:
  1. Order Service generates traceId when order is placed
  2. traceId is included in OrderEvent
  3. Inventory Service copies traceId to InventoryEvent
  4. Payment Service copies traceId to PaymentEvent
  5. All log lines include traceId: [traceId=abc-123] Processing payment

  With Zipkin/Jaeger:
  - Spring Cloud Sleuth generates trace/span IDs
  - KafkaTemplate and @KafkaListener auto-propagate trace context
  - Entire saga is visible as a single distributed trace

  spring:
    zipkin:
      base-url: http://zipkin:9411
    sleuth:
      sampler:
        probability: 1.0   # 100% sampling (dev), 0.1 (prod)
```

---

11. Interview Preparation Questions

11.1 Core Kafka Questions

Q1: What is Apache Kafka and why did you use it in your project?

Kafka is a distributed event streaming platform. We used it because our order processing system requires asynchronous communication between 6 microservices. When a user places an order, we need to: reserve inventory, process payment, and send notifications — all asynchronously. Kafka provides durable message storage, guaranteed

delivery, and enables loose coupling between services. If the Payment Service is temporarily down, Kafka retains the message until it recovers.

Q2: Explain the difference between a Topic, Partition, and Consumer Group.

A **Topic** is a named category for messages (e.g., "order-created"). A **Partition** is a subset of a topic — an ordered, immutable log of records. We use 3 partitions per topic for parallel processing. A **Consumer Group** is a set of consumers that cooperate to consume a topic. Within a group, each partition is consumed by exactly one consumer. This allows horizontal scaling — 3 instances of inventory-service each consume from one partition.

Q3: How do you ensure message ordering in Kafka?

Kafka guarantees ordering only within a single partition. We use `productId` as the partition key for `order-created`, `product-created`, and `order-cancelled` topics. Since `hash(productId) % partitions` always maps the same productId to the same partition, all events for a specific product are processed in order. This prevents race conditions where an order arrives before the product is initialized.

Q4: What is the difference between At-Least-Once and Exactly-Once delivery?

At-Least-Once: Message may be delivered more than once but never lost. The consumer commits the offset AFTER processing. If it crashes after processing but before commit, the message is redelivered. We use this with an idempotency table (ProcessedEvent) to handle duplicates.

Exactly-Once: Message is processed exactly once. Requires Kafka transactions, idempotent producers, and transactional consumers. More complex and lower throughput. We chose at-least-once + idempotency as the industry standard for e-commerce.

Q5: How does your system handle message failures?

We use Spring's `@RetryableTopic` annotation. When a consumer fails to process a message, it retries 3 times with exponential backoff (1s → 2s → 4s). After all retries are exhausted, the message is moved to a Dead Letter Topic (DLT), e.g., `order-created-dlt`. We also use `ErrorHandlingDeserializer` to handle poison messages (malformed JSON) without crashing the consumer.

Q6: What is the Dead Letter Topic (DLT) and how do you handle it?

A DLT is a dedicated Kafka topic for messages that failed processing after all retry attempts. Spring's `@RetryableTopic` with `DltStrategy.FAIL_ON_ERROR` automatically moves failed messages to a DLT named `<topic>-dlt`. In production, we would monitor DLT message counts and alert immediately. Failed messages can be investigated, fixed, and replayed to the original topic.

Q7: How do you handle duplicate message processing?

We use a `ProcessedEvent` database table with a unique constraint on `eventId`. Before processing any event, we check: `processedEventRepository.existsByEventId(eventId)`. If the event was already processed, we skip it. If not, we process it and insert the eventId atomically. This makes consumers idempotent — safe for at-least-once delivery.

11.2 Architecture Questions

Q8: Explain the Saga pattern in your project.

We implement a Choreography-based Saga. Each service makes local decisions and publishes events. For order placement: Order Service publishes `order-created` → Inventory Service reserves stock and publishes `inventory-updated` → Payment Service processes payment and publishes `payment-success` → Order Service updates final status. There's no central orchestrator — each service reacts to events autonomously.

Compensation: If inventory is insufficient, status = `INSUFFICIENT_STOCK`, order is cancelled. If payment fails, order status = `PAYMENT_FAILED`. If user cancels, `order-cancelled` triggers stock restoration (idempotent).

Q9: Why did you use a shared common-events module?

The common-events module provides a single source of truth for event schemas (DTOs). Without it, every service would need its own copy of OrderEvent, PaymentEvent, etc. — leading to version drift and deserialization failures. Every event includes an `eventId` (UUID) for idempotency, timestamps for ordering, and clear Javadoc documenting producer/consumer relationships.

Q10: How would you scale this system for 10x traffic?

1. Increase partitions per topic (3 → 12) to enable more parallel consumers
2. Scale consumer instances per service (1 → 12, one per partition)
3. Add Kafka brokers (1 → 5) with replication factor 3
4. Enable producer batching and compression (snappy)
5. Optimize consumer processing (batch DB writes, connection pooling)
6. Consider tiered storage for long-term message retention

Q11: What is the Outbox Pattern and when would you use it?

The Outbox Pattern solves the dual-write problem: when you need to write to a database AND publish to Kafka atomically. Instead of calling KafkaTemplate.send() after the DB write (which can fail independently), you write both the entity and an OutboxEvent to the SAME database transaction. A separate poller reads unpublished outbox events and sends them to Kafka. This guarantees at-least-once event publishing with zero message loss.

Q12: How does ErrorHandlingDeserializer prevent poison messages?

Without it: A malformed JSON message causes a `DeserializationException`. The consumer crashes on every retry — stuck at the same offset forever, blocking all subsequent messages.

With it: `ErrorHandlingDeserializer` wraps the `JsonDeserializer`. If deserialization fails, the error is caught and the message is delivered to the `@KafkaListener` with an error header. `@RetryableTopic` then sends it to the DLT after exhausting retries, allowing the consumer to continue processing the next message.

Q13: What is consumer rebalancing and how does it affect your system?

Consumer rebalancing occurs when consumers join/leave a group (deployment, crash, scaling). During rebalancing:

1. All partitions are revoked from all consumers
2. Partitions are redistributed among active consumers
3. No messages are processed during this window (seconds to minutes)

To minimize impact: Use `partition.assignment.strategy=CooperativeStickyAssignor` (incremental rebalancing). Also set `session.timeout.ms` and `heartbeat.interval.ms` appropriately to detect failures quickly but avoid false positives.

Q14: How do you monitor Kafka in production?

Key metrics: consumer lag (messages behind), throughput (messages/sec), error rate, DLT queue depth, broker health. Tools: Prometheus + Grafana for dashboards, Spring Boot Actuator + Micrometer for application metrics, `kafka-consumer-groups` CLI for real-time lag monitoring. Alert on: consumer lag > 1000 for 5 min, any DLT messages, under-replicated partitions.

---

12. Future Scalability Considerations

12.1 Schema Registry (Avro Migration)

```
CURRENT: JSON serialization (human-readable, no schema enforcement)
FUTURE:  Apache Avro + Confluent Schema Registry

BENEFITS:
  - Schema enforcement (producer rejects incompatible messages)
  - Backward/forward compatibility checks
  - Smaller message size (binary encoding, no field names)
  - Auto-generated Java classes from .avsc files

MIGRATION STEPS:
  1. Deploy Schema Registry alongside Kafka
  2. Define Avro schemas for each event type
  3. Add AvroSerializer/AvroDeserializer to Spring config
  4. Register schemas with compatibility mode (BACKWARD)
  5. Gradually migrate topics (new topics first, then existing)
```

12.2 Kafka Streams / ksqlDB

```
CURRENT: Simple produce-consume pattern
FUTURE:  Stream processing for real-time analytics

USE CASES:
  - Real-time order dashboard (orders per minute, revenue per hour)
  - Fraud detection (unusually high order amounts)
  - Inventory alerts (stock below threshold)

EXAMPLE (Kafka Streams):
  KStream<String, OrderEvent> orders = builder.stream("order-created");
  KTable<String, Long> orderCounts = orders
      .groupByKey()
      .count(Materialized.as("order-counts"));
  // Real-time count of orders per product
```

12.3 Multi-Cluster Replication (MirrorMaker 2)

```
CURRENT: Single Kafka cluster (development)
FUTURE:  Multi-region deployment

ARCHITECTURE:
  Region A (US-East): Primary Kafka Cluster
    ↓ MirrorMaker 2 (continuous replication)
  Region B (EU-West): DR Kafka Cluster

BENEFITS:
  - Disaster recovery (region failover)
  - Geo-locality (consumers read from nearest cluster)
  - Cross-region analytics
```

12.4 Kubernetes Deployment

```
CURRENT: Docker Compose (single host)
FUTURE:  Kubernetes (multi-node, auto-scaling)

COMPONENTS:
  - Strimzi Kafka Operator: Manages Kafka clusters on K8s
  - HPA (Horizontal Pod Autoscaler): Scale consumers based on lag
  - PersistentVolumeClaims: Durable storage for Kafka data
  - Service Mesh (Istio): mTLS between services and Kafka

EXAMPLE HPA:
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  metadata:
    name: inventory-service-hpa
  spec:
    scaleTargetRef:
      name: inventory-service
    minReplicas: 1
    maxReplicas: 6     # Match partition count
    metrics:
      - type: External
        external:
          metric:
            name: kafka_consumer_lag
            selector:
              matchLabels:
                consumer_group: inventory-service-group
          target:
            type: Value
            value: "1000"  # Scale up when lag > 1000
```

12.5 Event Catalog & Self-Service

```
FUTURE ENHANCEMENT: Event Catalog

A centralized registry of all events in the system:
  - Event name, schema, description
  - Producer service
  - Consumer services
  - SLA (latency, throughput)
  - Owner team

TOOLS:
  - EventCatalog.dev (open source)
  - AsyncAPI specification
  - Custom wiki/confluence pages

BENEFITS:
  - New developers understand event flows quickly
```

```
  - Service teams discover available events
  - Contract testing between producers and consumers
```

Appendix A — Configuration Reference

Complete Kafka Configuration (Order Service)

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.type.mapping: "orderEvent:com.orderprocessing.events.OrderEvent"
        enable.idempotence: true
      acks: all
      retries: 3
    consumer:
      group-id: order-service-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      properties:
        spring.deserializer.value.delegate.class:
org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.trusted.packages: "*"
        spring.json.type.mapping: >
          paymentEvent:com.orderprocessing.events.PaymentEvent,
          inventoryEvent:com.orderprocessing.events.InventoryEvent
```

Docker Compose — Kafka Environment Variables

Variable	Value	Purpose
SPRING_KAFKA_BOOTSTRAP_SERVERS	kafka:29092	Internal Docker network address
KAFKA_BROKER_ID	1	Unique broker identity
KAFKA_ZOOKEEPER_CONNECT	zookeeper:2181	ZooKeeper coordination
KAFKA_ADVERTISED_LISTENERS	PLAINTEXT://kafka:29092, PLAINTEXT_HOST://localhost:9092	Internal + external listeners
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR	1	Single broker dev setup

KAFKA_AUTO_CREATE_TOPICS_ENABLE	true	Auto-create topics on first use

---

Appendix B — Glossary

Term	Definition
EDA	Event-Driven Architecture — system design where events drive the flow
Broker	A Kafka server instance that stores and serves messages
Topic	A named stream of records; category for messages
Partition	An ordered, immutable log within a topic; unit of parallelism
Offset	Sequential ID of a message within a partition
Consumer Group	Set of consumers sharing topic consumption; each partition read by one consumer
ISR	In-Sync Replicas — follower replicas that are up-to-date with the leader
DLT	Dead Letter Topic — destination for messages that failed all retry attempts
@RetryableTopic	Spring annotation for automatic retry with exponential backoff
Idempotency	Property where processing a message multiple times yields the same result
Saga	Design pattern for managing distributed transactions across services
Outbox Pattern	Technique for atomically writing to DB and publishing to Kafka
CQRS	Command Query Responsibility Segregation — separate read and write models
Event Sourcing	Storing state as a sequence of events rather than current state
ACL	Access Control List — Kafka authorization rules per topic/consumer
KRaft	Kafka Raft — new consensus protocol replacing ZooKeeper
ErrorHandlingDeserializer	Spring wrapper that catches deserialization errors gracefully
ProcessedEvent	Database table tracking consumed events for idempotency
Backpressure	When producer sends faster than consumer can process

---

End of Document

This guide provides a comprehensive, enterprise-level reference for building Event-Driven Architecture with Apache Kafka in Spring Boot Microservices. For implementation, start with Step 1 (Kafka Setup) and follow the steps sequentially. Each section is self-contained and can be referenced independently.